Practical No. 5

Mapped CO: CO3 Units V Bloom Level: L6 – Create

1.0) AIM

To develop a Web3-based application that connects to a smart contract and performs contract interaction using Web3.js.

12.0) SCOPE

This practical covers:
- Setting up a Node.js environment with Web3.js library
- Connecting to a local Ethereum blockchain (Ganache)
- Creating a contract instance using contract address and ABI
- Understanding the difference between GUI-based (Remix) and programmatic (Web3.js) contract interaction

3.0) FACILITIES / APPARATUS

Software Requirements:

- **Node.js** v14 or higher (v22.21.1 used in this practical)
- **npm** (Node Package Manager) - comes with Node.js
- **Ganache v2.7.1** - Local blockchain emulator
- **Remix IDE** (https://remix.ethereum.org) - For smart contract deployment
- **VS Code** or any text editor
- **Web3.js** library v4.x

Internet connection (for Remix IDE access)

Libraries to Install:

```
npm install web3

npm install prompt-sync
```

4.0) THEORY

4.1 Web3.js Library

Web3.js is a JavaScript library collection that allows developers to interact with Ethereum blockchain and smart contracts. It provides:

- **Connection to blockchain nodes** (Ganache)
- **Contract instance creation** and interaction
- **Account and wallet management**
- **Transaction monitoring** and status tracking
- **Event listening** from smart contracts
- **Utility functions** for encoding/decoding data

4.2 Key Concepts

Provider

A provider is the connection point to the blockchain. It handles all communication between your application and the blockchain node.

```
javascript
const web3 = new Web3('http://localhost:7545');
```

- `http://localhost:7545` = Ganache running on local machine
- Like a **URL to the blockchain server**

Contract ABI (Application Binary Interface)

The ABI is a JSON array that defines:

- What functions exist in the contract
- What parameters each function needs
- What data types are returned
- Whether function is `view` (read-only) or `nonpayable` (state-changing)

Without ABI, Web3.js doesn't know how to call your contract functions.

Contract Instance

Combines the ABI and contract address to create an object that represents the deployed contract:

```
javascript
const contract = new web3.eth.Contract(contractABI, contractAddress);
```

Call vs Send

Operation	Purpose	Gas Cost	Returns Data
`.call()`	Read-only operations (view functions)	FREE (0 gas)	Yes
`.send()`	State-changing operations (write functions)	PAID (⛽ gas)	Transaction hash

4.3 Transaction Flow

1. User Input
↓
2. Web3 prepares function call
↓
3. Sends to Ganache blockchain
↓
4. Ganache processes transaction
↓
5. Updates contract state

↓

6. Creates new block

↓

7. Returns transaction hash & gas used

↓

8. Application displays results

4.4 Gas and Transactions

Gas: Unit of computational work required to execute a transaction on blockchain.

- **Read operations (call):** 0 gas (free)
- **Write operations (send):** Variable gas (costs ether)
- **Gas Price:** Amount of ETH per gas unit
- **Gas Limit:** Maximum gas allowed for a transaction

5.0) Smart Contract Used (Solidity — SimpleStorage.sol Practical 3 code)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage
{
    // State variable: stored permanently on the blockchain
    uint256 storedNumber;

    // set() writes a new value (a transaction)
    function set(uint256 newNumber) public
     {
        storedNumber = newNumber;
     }

    // get() reads the stored value (free, no transaction)
    function get() public view returns (uint256)
     {
        return storedNumber;
     }
}
```

Functions:

- `set(uint256)` - Writes a number to blockchain (costs gas)
- `get()` - Reads the number from blockchain (free, no gas)

6.0) PROGRAM

```
const {Web3} = require('web3');
const prompt = require('prompt-sync')();

// Connect to Ganache
const web3 = new Web3('http://localhost:7545');
```

```
// Contract details
const contractAddress = 'XXXXXXXXXXXXXXXX'; // Replace with your deployed contract address
const contractABI =
[
];// Replace with your contract's ABI
```

```
// Create contract instance
const contract = new web3.eth.Contract(contractABI, contractAddress);
```

```
// Function to get first account from Ganache
async function getAccount()
 {
  const accounts = await web3.eth.getAccounts();
  console.log('Using Account:', accounts[0]);
  return accounts[0];
}
```

```
// Function to read value (call - no gas)
async function getValue() {
  Try
 {
    const result = await contract.methods.get().call();
    console.log(' Current Value:', result);
    return result;
  } catch (error)
 {
    console.error(' Error reading value:', error.message);
  }
}
```

```
// Function to set value (send transaction - costs gas)
async function setValue(newValue) {
  try {
    const account = await getAccount();
    console.log('Sending transaction to set value to:', newValue);

    const tx = await contract.methods.set(newValue).send({
      from: account,
      gas: 100000
    });

    console.log(' Transaction Hash:', tx.transactionHash);
    console.log(' Gas Used:', tx.gasUsed);
    console.log(' Block Number:', tx.blockNumber);
    return tx;
  }
catch (error)
 {
    console.error(' Error setting value:', error.message);
  }
}
```

```
// Main function
async function main() {
  console.log('=== Web3 Contract Interaction ===\n');

  // STEP 1: Read initial value
  console.log('STEP 1: Reading initial value...');
```

```
  await getValue();

  // STEP 2: Get user input
  console.log('\nSTEP 2: User Input');
  const userValue = prompt('Enter value to set: ');
  console.log('Setting new value to:', userValue);
  await setValue(userValue);

  // STEP 3: Read updated value
  console.log('\nSTEP 3: Reading updated value...');
  await getValue();

  console.log('\n=== Test Complete! ===');
}
```

```
main().catch(console.error);
```

7. PROCEDURE / STEPS TO EXECUTE

Step 1: Install Node.js and NPM

1. Download Node.js from https://nodejs.org
2. Install with default settings
3. Verify installation:

bash

```
node --version
npm --version
```

Step 2: Start Ganache

1. Open Ganache v2.7.1
2. Click "Quickstart"
3. Wait for blockchain to start
4. Note the address: `http://127.0.0.1:7545`
5. Ganache should show 10 accounts with 100 ETH each

Step 3: Deploy Smart Contract in Remix

1. Go to https://remix.ethereum.org
2. Create new file: `SimpleStorage.sol`
3. Copy the smart contract code (Section 5)
4. Go to "Solidity Compiler" tab
5. Change compiler version to **0.8.19**
6. Click "Compile"
7. Go to "Deploy & Run Transactions" tab
8. Change Environment to "Custom - External Http Provider"
9. Enter URL: `http://127.0.0.1:7545`
10. Click "OK"
11. Select contract and click "Deploy"
12. Copy the **contract address** and **ABI** from deployed contracts

Step 4: Set Up Node.js Project

1. Open Terminal/Command Prompt
2. Create project folder:

```
bash
mkdir web3-app
cd web3-appnpm init -y
```

1. Install dependencies:

```
bash
npm install web3
npm install prompt-sync
```

Step 5: Create interact.js

1. Create file: `interact.js`
2. Paste the code from Section 6
3. Replace placeholder values:
    a. `contractAddress` = your deployed contract address
    b. `contractABI` = your contract's ABI (copy from Remix)

Step 6: Run the Application

```
bash
node interact.js
```

Step 7: Provide User Input

When prompted:

```
Enter value to set: 20
```

Enter any number and press Enter

Step 8: Verify in Ganache

1. Open Ganache "Blocks" tab
2. Verify new blocks were created
3. Check gas usage for transactions

8.0) EXPECTED OUTPUT

```
C:\Users\Pallavi Bute Mankar\web-app>node interact.js
=== Web3 Contract Interaction ===

STEP 1: Reading initial value...
 Current Value: 5n

STEP 2: User Input
Enter value to set: 2
Setting new value to: 2
Using Account: 0x61e15fED5b7260757926102F99B2EBb1579a61C3
Sending transaction to set value to: 2
 Transaction Hash: 0x231c1d2f42a303b52f49dd7155ea0197996d1045fa5b1a9cf97500bf8fbeb25c
 Gas Used: 26602n
 Block Number: 3n

STEP 3: Reading updated value...
 Current Value: 2n

=== Test Complete! ===

C:\Users\Pallavi Bute Mankar\web-app>
```

9.0) CONCLUSION

Practical 5 successfully demonstrated how to build the backend of a decentralized application using Web3.js. Students learned that Web3.js acts as a bridge connecting JavaScript code to smart contracts on the blockchain, enabling read operations (free) and write operations (with gas cost).